

本科科研预备 · 实施手册（已按本机更新）

眼在手上机械臂 视觉伺服闭环控制 实施手册

ROS 1 Noetic · 实验室 catkin 工作空间 robot_gripper

含模块 S：启动机械臂并下发运动指令

工作空间：/home/biand/workspace_ws/src/robot_gripper_9_91/robot_gripper_5_29/robot_gripper

包：arm_description · arm_moveit_config · abb_driver · control_robot · vi_grab · bio_ik

启动脚本：start.sh · start_ui.sh · launch_synarm_ui.sh

应用层：Python / moveit_commander · 不改 abb_driver · 不做 ROS 2 课设仿真

顺序：模块 S（启动 + RViz / Python 运动）→ A → B → C → D → E → F → G → H

这一模块先于步骤 A ~ H。臂还不能按你的指令动，后面的检测和伺服都不要做。

本机版本（已由你贴出的 listing 锁定，不是课设 ROS 2）：

项	值
系统	WSL Ubuntu 20.04
ROS	1 Noetic（/opt/ros/noetic）
工作空间根	/home/biand/workspace_ws/src/robot_gripper_9_91/robot_gripper_5_29/robot_gripper
构建	catkin_tools（devel/，不是 install/）
臂描述 / 规划	src/arm_description + src/arm_moveit_config
真机驱动	src/abb_driver（不要改）
组里封装	src/control_robot、src/robot_serial
IK	src/bio_ik（MoveIt 插件，调用即可）
视觉 / UI	src/vi_grab、src/robot_ui（本模块先不管相机）
实验室启动脚本	工作空间根下的 start.sh、start_ui.sh、launch_synarm_ui.sh

下文命令里的 \$WS 一律等于上面那一行工作空间根。先在每个新开的终端执行一次：

```
export WS=/home/biand/workspace_ws/src/robot_gripper_9_91/robot_gripper_5_29/robot_gripper
source /opt/ros/noetic/setup.bash
source "$WS/devel/setup.bash"
cd "$WS"
```

也可以：source /path/to/本仓库/vs_practice/env.sh

S.0 目的、原理、验收

目的

学会四件事，并且能口头复述：

1. 在哪一层目录启动（最内层 robot_gripper，不是 workspace_ws）。
2. 实验室三条脚本各自开什么，不要同时开两套 bringup。
3. 用 RViz MotionPlanning Execute 发出第一条轨迹。
4. 用 moveit_commander 发出第二条小位移（Python，不是改 driver）。

原理（本机这条链）

```
你（RViz 或 Python Commander）
  → MoveIt 1（arm_moveit_config / move_group，IK 可能走 bio_ik）
    → FollowJointTrajectory（controllers.yaml 里的名字）
      → abb_driver / control_robot
        → 控制器 / 仿真
          → /joint_states、/tf 回来
```

- start.sh 一类脚本负责把 驱动 + MoveIt + 必要节点 拉起来。
- RViz 的 Execute 和 Python 的 arm.go() 是同一层：都是给 MoveIt 下目标，不是直接转电机。
- abb_driver 是厂家/工业接口，本练习只调用、不改。

验收（本模块完成线）

- /joint_states 有数且在刷新。
- RViz 里规划组能选中，拖一次末端，Plan 成功且 Execute 后关节角变化。
- rosrn vs_practice send_small_motion.py 再让末端动约 2 cm（或明确打印 dry-run 目标）。
- 你能一句话说明：谁规划、谁跟踪轨迹、谁是 driver。

S.1 安全（真机必读，仿真也建议照做）

1. 工作空间清空，急停位置你够得到。组里有示教器/急停键，先确认谁按。
2. 第一条运动只允许 很小：笛卡尔 ≤ 4 cm，速度缩放 0.1。
3. 终端里随时：

```
rostopic pub /vs/cmd std_msgs/String "data: stop"
```

或在跑 send_small_motion.py 的终端 Ctrl+C。真机以组里急停为准。

1. 不要两个终端同时 ./start.sh 和 ./start_ui.sh（会抢 roscore / 驱动）。
2. 不要对 /home/biand/workspace_ws 跑 catkin build 或 colcon。

S.2 先读实验室脚本（30 秒，不要猜）

在已 source 的终端：

```
cd "$WS"
ls -l start.sh start_ui.sh launch_synarm_ui.sh
echo "==== start.sh ====="
sed -n '1,80p' start.sh
echo "==== start_ui.sh ====="
sed -n '1,80p' start_ui.sh
echo "==== launch_synarm_ui.sh ====="
sed -n '1,80p' launch_synarm_ui.sh
echo "==== MoveIt launch ====="
ls src/arm_moveit_config/launch
```

对着脚本正文，用下表判断你走哪条路径：

脚本里主要在 launch 什么	本模块用它来
arm_moveit_config 的 demo.launch / fake / sim:=true	路径 1：不上真机，只练 MoveIt + Commander
abb_driver / robot_ip / control_robot	路径 2：真机或实验室标准 bringup → 优先 ./start.sh
robot_ui / launch_synarm_ui	路径 3：要实验室 UI 时用；运动仍由 MoveIt 执行
vi_grab / RealSense / JEPA	本模块先不管；等步骤 C 再开相机

把 `ls src/arm_moveit_config/launch` 里实际文件名记下来。常见会有 `demo.launch`、`moveit_planning_execution.launch`。以你磁盘上的名为准。

S.3 启动（三条路径只开一条）

路径 1 — 只核对 MoveIt（不上真机、不抢实验室 UI）

适合：第一次练「下发运动」，控制器不在旁边。

```
# 终端 1（保持不关）
export WS=/home/biand/workspace_ws/src/robot_gripper_9_91/robot_gripper_5_29/robot_gripper
source /opt/ros/noetic/setup.bash
source "$WS/devel/setup.bash"
roslaunch arm_moveit_config demo.launch
```

若提示找不到 demo.launch，改成 S.2 里看到的那个带 demo / fake / rviz 的 launch。成功时一般会弹出 RViz，左侧有 MotionPlanning。

路径 2 — 实验室日常（优先）

适合：组里平时就是这样开臂。

```
# 终端 1
export WS=/home/biand/workspace_ws/src/robot_gripper_9_91/robot_gripper_5_29/robot_gripper
source /opt/ros/noetic/setup.bash
source "$WS/devel/setup.bash"
cd "$WS"
./start.sh
```

真机时还要满足组里规程（控制器 AUTO、电机使能、RAPID/通信程序在跑、IP 通）。这些写在你们的 ROS_DOCUMENTATION.md 里，本手册不替代。

若 start.sh 没带 RViz，再开终端 2：

```
export WS=/home/biand/workspace_ws/src/robot_gripper_9_91/robot_gripper_5_29/robot_gripper
source /opt/ros/noetic/setup.bash
source "$WS/devel/setup.bash"
roslaunch arm_moveit_config moveit_rviz.launch
```

moveit_rviz.launch 若没有，用 S.2 列出的带 rviz 的文件，或在已有 RViz 里手动加 MotionPlanning 插件。

路径 3 — 要实验室 UI（SynArm / robot_ui）

```
cd "$WS"
# 二选一，不要和路径 2 同时开
./start_ui.sh
# 或
./launch_synarm_ui.sh
```

UI 可以留着。本模块的运动指令仍然走 RViz / send_small_motion.py，不要在 UI 里乱点和大位移。

S.4 确认「臂已经在听」（另开终端 2）

```
export WS=/home/biand/workspace_ws/src/robot_gripper_9_91/robot_gripper_5_29/robot_gripper
source /opt/ros/noetic/setup.bash
source "$WS/devel/setup.bash"
```

```
echo "ROS_DISTRO=$ROS_DISTRO"      # 必须是 noetic
rostopic echo /joint_states -n 1    # 必须有 position 数组
rostopic hz /joint_states           # Ctrl+C 停；应有稳定 Hz
rostopic list | grep -E 'joint|move_group|follow|traj|camera'
rosservice list | grep -E 'plan|execute|controller|follow' | head
```

现象	含义	怎么办
Unable to communicate with master	终端 1 没起来或没 source	先看终端 1 有没有报错；每个终端都要 source devel
没有 /joint_states	驱动或 fake joint 没起	回到 S.2 看脚本少 launch 了哪一段
有 /joint_states 但没有 move_group	只开了驱动，没开 MoveIt	补 roslaunch arm_moveit_config ... (S.2 里的文件)
ROS_DISTRO 不是 noetic	source 错了发行版	只 source /opt/ros/noetic

可选（若已拷入 vs_practice）：

```
roslaunch vs_practice print_interface.py
```

记下打印的 planning groups 和 ee=。后面 --group 必须用这个名字，不要写 panda_arm。

S.5 用 RViz 下发第一条运动指令

这是本机「运动指令」的第一种发法：人在界面里给目标，MoveIt 规划，轨迹控制器执行。

1. RViz 左侧找到 MotionPlanning。没有就：Panels → Add New Panel → MotionPlanning。
2. Context / Planning 里 Planning Group 选 print_interface.py 打出来的臂组（常见 manipulator，以打印为准）。
3. 勾选 Query 的 Start 等于当前状态（或点 Update / Current）。
4. 用橙色/交互标记把末端平移 很小一截（目测 2 ~ 3 cm）。不要转很大关节。
5. 点 Plan。成功则出现一条轨迹；失败先换近一点的目标，不要改 abb_driver。
6. 速度缩到很低（若有 Velocity Scaling，先 0.1）。
7. 点 Execute。看真机或 RViz 里的臂跟着走。
8. 终端 2：再执行一次 `rostopic echo /joint_states -n 1`，确认数字相对步骤 S.4 变了。

验收：Execute 成功 + 关节值变化。失败只停在本步。

把这次成功的末端位姿抄下来，以后写入 `vs_practice/config/waypoints.yaml`。在终端 2 也可以等 S.6 的脚本帮你打印当前位姿。

S.6 用 Python 下发第二条运动指令

这是本机「运动指令」的第二种发法：和课程第八章同一层，但是 `MoveIt 1 / rospy / moveit_commander`，不要拷 ROS 2 节点。

先确认 `vs_practice` 已在 `$WS/src/vs_practice` 并 `catkin build vs_practice`、重新 `source devel/setup.bash`。

只看当前状态（臂不动）：

```
roslaunch vs_practice send_small_motion.py _group:=manipulator _dry_run:=true
```

把 `_group` 换成 S.4 打印的组名。屏幕会打出当前关节、当前末端位姿（可抄进 `waypoints.yaml`）。

真的动 2 cm（沿当前末端 z，先 dry-run 看目标再执行）：

```
# 先看目标，不动
roslaunch vs_practice send_small_motion.py _group:=manipulator _dz:=0.02 _dry_run:=true
# 确认目标合理后再动（≤ 4 cm）
roslaunch vs_practice send_small_motion.py _group:=manipulator _dz:=0.02 _dry_run:=false
```

或：

```
roslaunch vs_practice send_small_motion.launch group:=manipulator dz:=0.02 dry_run:=false
```

脚本内部做的事：

1. `MoveGroupCommander(group)` 接到已经在跑的 `move_group`。
2. `get_current_pose()` 取当前末端。
3. 位置加 `dx,dy,dz`（默认只加 `dz=0.02`）。
4. `set_max_velocity_scaling_factor(0.1)`。
5. `set_pose_target` → `go(wait=True)` → `stop` → `clear_pose_targets`。

位移绝对值超过 4 cm 会直接拒绝。这就是「调用 Commander」，不是写 driver。

验收：日志 `go ok=True`；`/joint_states` 再变一次；你能指出代码里没有改 `abb_driver`。

S.7 停机

```
# 停练习节点  
pkill -f send_small_motion.py  
# 停实验室 bringup: 回到终端 1, Ctrl+C  
# 真机: 按组里急停 / 示教器停止, 再断电规程
```

不要在臂还在动时关 WSL 窗口。

S.8 运动指令失败对照

现象	先查
No move_group / Commander 超时	终端 1 是否 launch 了 arm_moveit_config
Invalid group name	--group / _group 与 SRDF 不一致，跑 print_interface.py
Plan 失败	目标太远、碰撞、超限；先减到 2 cm
Plan 成功但臂不动	假执行 vs 真机：路径 1 只动 RViz；真机要路径 2 且控制器使能
一启动就报 robot_ip / socket	abb_driver 连不上控制器；查 ROS_DOCUMENTATION.md 和网线，不改驱动源码
UI 和 RViz 抢臂	只留一条 bringup；UI 打开时不要再 ./start.sh

S.9 做完之后

本模块通过 → 进入 implementation_guide.md 步骤 A (填 interface.yaml)。

步骤 A 的「RViz 拖一次」若在 S.5 已做过，可以只补 print_interface.py 的登记。

步骤 B 把 S.6 的单次 go() 扩成 APPROACH → DESCEND 流程。

文件性质：分步实施指导书（目的—原理—实现—验收）

本机栈（已由 listing 锁定）：WSL Ubuntu 20.04 · ROS 1 Noetic · catkin_tools

工作空间根：/home/biand/workspace_ws/src/robot_gripper_9_91/robot_gripper_5_29/robot_gripper

仿真/真机模型：只用该工作空间里的 arm_description + arm_moveit_config，不用课设 URDF，不另起 ROS 2 仿真

语言约定：应用层 Python；调用 moveit_commander；不写 Commander 库、不改 abb_driver

对照清单见 NOETIC_ADJUST.md。

启动臂、RViz / Python 下发运动（先做这一章）：START_ARM.md

下文每一步：目的、原理、实现、验收。未通过验收不得进入下一步。顺序：

模块 S（启动 + 运动指令）→ A → B → C → D → E → F → G → H

0. 项目在做什么（实施视角）

做出一条与现有 vi_grab 「检测一次再规划」不同的链路：

1. 用 MoveIt 1 Commander 把末端送到相机能看见 ArUco 的位置（开环、粗、快）。
2. 停止规划长轨迹，每一帧用标签中心相对图像中心的像素误差生成运动量。
3. 运动量按本机接口写入臂：若有关节速度话题则走速度；ABB/abb_driver 通常只有轨迹接口，则走几毫米笛卡尔增量。臂一动，下一帧误差就变，形成闭环。
4. 用表格比较开环与闭环；再给图像加延迟、让标签慢速移动，比较三种控制律。

视觉只提供误差。思考在控制：增益、限幅、延迟、停机。现有 UI / JEPA 采集线不要混进来。

新建包：src/vs_practice（已在本仓库写好）。规划组名以本机 SRDF / print_interface.py 为准。

```
vi_grab 或 RealSense 已有图像 --> vs_detect --> /vs/error, /vs/tag_ok
/joint_states, /tf --> vs_servo --> 速度或短笛卡尔增量
vs_fsm --APPROACH--> vs_commander --> 轨迹控制器（abb_driver / MoveIt）
vs_fsm --SERVO--> vs_servo
```

步骤 A 环境确认与信息登记

目的

避免在规划组、图像话题、控制器接口都未知时写代码。本步只产生 interface.yaml。

原理

后续节点要写死：规划组、关节顺序、图像话题、servo_mode。这些来自 arm_moveit_config、vi_grab、control_robot，不是课设 panda 名。

实现

1. 模块 S 已经通过（臂能 Execute，Python 能发 2 cm）。
2. 把本仓库 vs_practice/ 拷进最内层 src/（若 S.6 还没拷），在该层 catkin build vs_practice, source devel/setup.bash。不要对 /home/biand/workspace_ws 做 colcon / catkin。
3. 臂保持模块 S 的启动方式，不要另开第二套 bringup。另开终端：

```
source /opt/ros/noetic/setup.bash
source /home/biand/workspace_ws/src/robot_gripper_9_91/robot_gripper_5_29/robot_gripper/devel/setup.bash
roslaunch vs_practice print_interface.py
rostopic echo /joint_states -n 1
```

1. 打开 src/arm_moveit_config 的 SRDF 与 controllers.yaml，把打印结果写入 vs_practice/config/interface.yaml。
2. 模块 S 里成功的那次末端位姿写入 waypoints.yaml（不要抄手册示例数字）。

验收

清单填满。RViz / Python 运动在模块 S 已验收。没有 controller_manager 不算失败——ABB 经常没有，记到 yaml 即可。

步骤 B 用 Python 调用 Commander 完成开环流程

目的

把「会调用」落成状态机骨架。以后视觉只替换目标从哪来。

原理

moveit_commander.MoveGroupCommander 是 MoveIt 1 客户端。set_pose_target + go() 请规划器出轨迹，由 abb_driver / 轨迹控制器跟踪。调用者不是驱动器。bio_ik 若已写在 kinematics.yaml，会在规划时自动用，不要直接调它的 C++。

状态：APPROACH → DESCEND → GRASP → LIFT → DONE。无夹爪则省略 GRASP。

实现

1. 在 RViz 拖出安全位姿，写入 vs_practice/config/waypoints.yaml（禁止沿用手册里的示例数字）。
2. 确认 arm_group 与 print_interface.py 一致后：

```
roslaunch vs_practice commander_only.launch arm_group:=<你的组名>
```

1. 看 /vs/commander_state。规划失败先改 yaml。
2. rosbag record /joint_states /tf。连续自动跑两遍。

验收

不手动拖目标，脚本能走完全流程。不要改 abb_driver。

步骤 C 眼在手相机与 ArUco 检测

目的

让图像成为系统输入。本步只做测量。

原理

ArUco 四角点取中心。眼在手相机固连末端。光学系与法兰系不同。

实现

1. 不要再 `roslaunch realsense2_camera` 第二份。 `rostopic list | grep -i image`，把 `vi_grab / RealSense` 已有话题写入 `interface.yaml`。
2. 打印或放置 ArUco (DICT_4X4_50，边长约 0.05 m)。
3. `roslaunch vs_practice detect_only.launch image_topic:=<实际话题>`。
4. 看 `/vs/tag_ok`、`/vs/error`，可用 `image_view` 看 `/vs/debug_image`。
5. `roslaunch tf_echo <ee_link> <camera_optical_frame>`。

验收

看见为真，遮挡为假。禁止把像素送给 MoveIt。

步骤 D 开环视觉引导

目的

图像只用一次生成位姿，交给 Commander，作为开环基线。

原理

$T_{base_tag} = T_{base_ee} * T_{ee_cam} * T_{cam_tag}$ ，只在 APPROACH 入口算一次。运动中途标签再动，轨迹不变。

实现

`roslaunch vs_practice vs_practice.launch mode:=open`，然后 `rostopic pub /vs/start std_msgs/String "data: go"`。到位后再读像素，记开环终值误差。

验收

能说明图像只用了一次。本关不把误差送进伺服。

步骤 E 静态视觉闭环（核心）

目的

停止规划长轨迹，误差生成运动。这是与现有抓取流程的分界。

原理

误差 $e = (u - cx, v - cy)$ 。简化律 $v_x = -\lambda Z e_u / f_x$ ， v_y 同理。看丢则运动清零。伺服段禁止再 `plan()` 一条自由空间长轨迹。

接口（本机必须先查）：

- 有关节速度话题 → `servo_mode: joint_velocity`
- 只有 ABB 轨迹（常见）→ `servo_mode: cartesian_increment`（每周期几毫米）。 λ 更小。不要用 2 Hz 大位移点冒充闭环。

实现

1. 填好 `interface.yaml` 的 `servo_mode`。
2. `roslaunch vs_practice vs_practice.launch mode:=closed`
3. `rostopic pub /vs/start std_msgs/String "data: go"`
4. λ 从很小加到刚振荡再回调。

验收

伺服段无一次长轨迹 `plan()`；遮挡后运动为 0。本步是进组演示最低完成线。

步骤 F 开环 / 闭环对照

同一初始条件，开环与闭环各 ≥ 5 次。rosv bag record /joint_states /tf /vs/error /vs/state。计算准（稳态像素误差）、快（调节时间）、稳（振荡/超调）。缺表不算完成。

步骤 G 运动目标、延迟与三种律

步骤 E 稳定后：标签慢速移动；image_delay_ms 取 50/100/150；law:=p|gain_schedule|predict。只替换 image_law()。

步骤 H 真机

本机已经在实验室包上。先复现步骤 E。占机短、调参不在臂上盲加 λ 。急停按组里规程。不改 `abb_driver`。

进度

S 启动+运动指令 → A 清单 → B Commander 流程 → C 检测 → D 开环视觉 → E 速度/增量闭环 ★最低完成线 → F 对照 → G 延迟三律 → H 真机细则

当前：栈已锁定为 Noetic 实验室包。下一步做 模块 S (START_ARM.md)，不要先写检测。